Teste e Validação de Software

Segundo Projeto — Relatório

Dinis Rodrigues (ist106213) Artur Krystopchuk (ist104145)
Gabriel Santos (ist1116334)

Junho 2026

Resumo

Este relatório apresenta o desenho de casos de teste para o sistema de entregas urbanas descrito no enunciado do segundo projeto da disciplina Teste e Validação de Software. São aplicadas estratégias de desenho apropriadas para testar as classes Order e Delivery ao nível de classe, bem como os métodos `computeDeliveryCost()` e `updateItem()`. A última secção apresenta a concretização em TestNG de oito casos de teste seleccionados da bateria de testes da classe Order.

Índice

1 Testes ao nível de classe da classe Order **2**
1.1 Estratégia escolhida: Teste de Classe Não-Modal . . . 2
1.2 Pontos On/Off . . . 2
1.3 Matriz de domínio com vetores de teste . . . 2
1.4 Invariante relacional (mesmo fornecedor) . . . 4

2 Testes ao nível de classe da classe Delivery **4**
2.1 Máquina de estados . . . 4
2.2 Transições válidas e inválidas . . . 4
2.3 Estratégia: *Round-Trip / Transition Coverage* . . . 5
2.4 Casos de teste . . . 5

3 Método `computeDeliveryCost()` **5**
3.1 Estratégia: Decision Table / Boundary Analysis . . . 5
3.2 Tabela de decisão . . . 5
3.3 Casos de teste . . . 5

4 Método `updateItem()` **5**
4.1 Estratégia escolhida . . . 5
4.2 Combinações estado + entrada . . . 5
4.3 Casos de teste . . . 5

5 Concretização em TestNG **5**

1 Testes ao nível de classe da classe Order

1.1 Estratégia escolhida: Teste de Classe Não-Modal

A classe Order não possui uma máquina de estados (não há modos que alterem o comportamento dos métodos); o seu comportamento ao nível de classe é determinado por um conjunto de *invariantes* sobre as suas variáveis de instância.

A estratégia apropriada para testar uma classe deste tipo é o **Teste de Classe Não-Modal**, que consiste em: (i) identificar os domínios das variáveis; (ii) determinar os pontos *On*/*Off* de cada fronteira de invariante; e (iii) construir uma *matriz de domínio* com vetores de teste, em que cada caso de teste exercita uma fronteira de cada vez, mantendo as restantes variáveis em valores típicos.

1.2 Pontos On/Off

Fronteira (invariante)	**On**	**Off**
client $\neq$ `null`	Cliente válido	`null`
addr $\neq$ `null`	String válida	`null`
addrLength ≥ 1	1	0 (vazio)
distance ≥ 0	0	-1
distance ≤ 20000	20000	20001
quantity ≥ 1	1	0
quantity ≤ 10	10	11
cost ≤ 1000	1000	1000.01
totalQty $\leq 20 + \lfloor \text{cost}/50 \rfloor$	= limite	limite + 1

1.3 Matriz de domínio com vetores de teste

Cada coluna representa um caso de teste. Em cada coluna, exatamente uma fronteira está em *On* ou *Off* (célula sombreada), estando as restantes variáveis num valor típico (linha *In*).

Valor típico adotado: encomenda com um item de preço 50 e quantidade 4 (cost = 200, totalQty = 4, limite = 24), morada de comprimento 3 e distance = 5000.

Tabela 2: Matriz de domínio para a classe Order.

Var.	Condição		1	2	3	4	5	6	7	8	9	10
addr	≥ 1	On	1									
		Off		0								
	Típico	In			3	3	3	3	3	3	3	3
distance	≥ 0	On			0							
		Off				-1						
	≤ 20000	On					20000					
		Off						20001				
	Típico	In	5000	5000					5000	5000	5000	5000
quantity	≥ 1	On							1			
		Off								0		
	≤ 10	On									10	
		Off										11
	Típico	In	4	4	4	4	4	4				
cost	≤ 1000	On										
		Off										
	Típico	In	200	200	200	200	200	200	50	0	500	550
totalQty	$\leq$ lim.	On										
		Off										
	Típico	In	4	4	4	4	4	4	1	0	10	11
Resultado esperado			✓	✗	✓	✗	✓	✗	✓	✗	✓	✗

Tabela 3: Matriz de domínio para a classe Order (continuação).

Var.	Condição		11	12	13	14	15	16	17	18
client	$\neq$ `null`	On					ok			
		Off						`null`		
	Típico	In	ok	ok	ok	ok			ok	ok
addr	$\neq$ `null`	On							`"x"`	
		Off								`null`
	compr. típico	In	3	3	3	3	3	3		
distance	Típico	In	5000	5000	5000	5000	5000	5000	5000	5000
quantity	Típico	In	—	—	—	—	—	—	—	—
cost	≤ 1000	On	1000							
		Off		1000.01						
	Típico	In			200	200	200	200	200	200
totalQty	$\leq$ lim.	On			24					
		Off				25				
	Típico	In	20	20			4	4	4	4
Resultado esperado			✓	✗	✓	✗	✓	✗	✓	✗

Legenda: ✓ = estado válido (operação aceite); ✗ = estado inválido (deve lançar `InvalidOperat`

`ionException` sem alterar o estado da encomenda, conforme I6).

1.4 Invariante relacional (mesmo fornecedor)

Este invariante não é numérico, pelo que não tem pontos On/Off. Testa-se com dois casos dedicados:

#	Cenário	Resultado esperado
19	Adicionar item de fornecedor S a encomenda já com itens do fornecedor S	Aceite (✓)
20	Adicionar item de fornecedor S' a encomenda já com itens do fornecedor S	`InvalidOperationException` (✗)

2 Testes ao nível de classe da classe **Delivery**

2.1 Máquina de estados

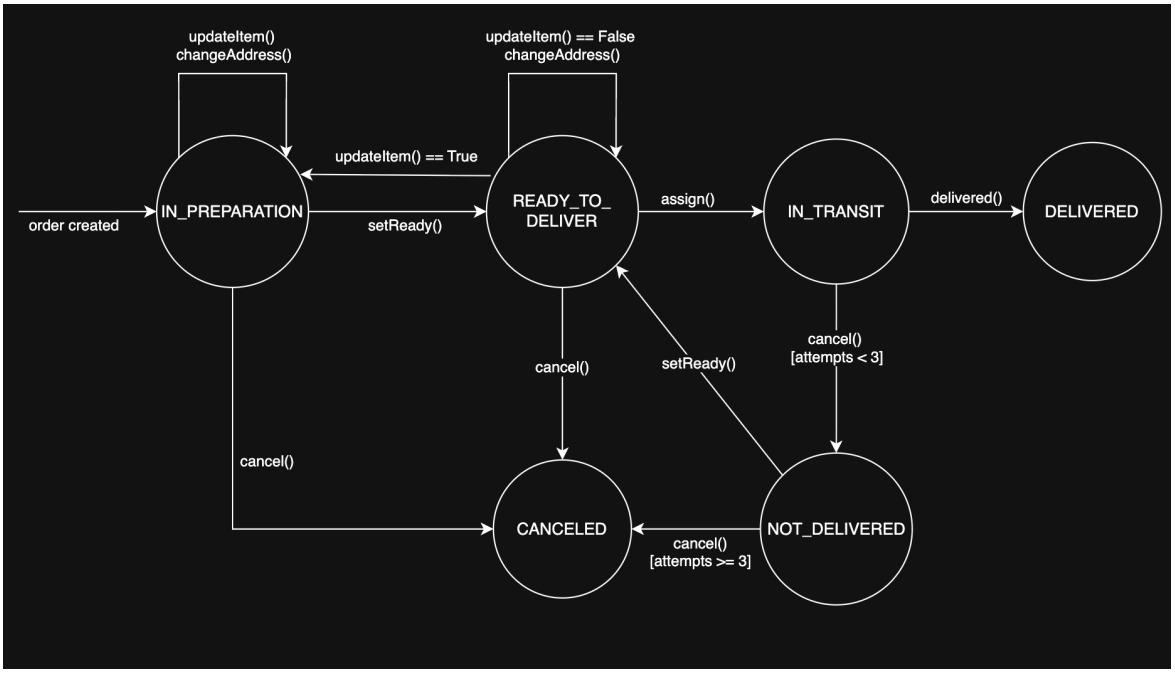


Figura 1: Máquina de estados da classe Delivery.

2.2 Transições válidas e inválidas

Tabela de transições a preencher.

2.3 Estratégia: *Round-Trip / Transition Coverage*

2.4 Casos de teste

3 Método `computeDeliveryCost()`

3.1 Estratégia: Decision Table / Boundary Analysis

3.2 Tabela de decisão

3.3 Casos de teste

4 Método `updateItem()`

4.1 Estratégia escolhida

4.2 Combinações estado + entrada

4.3 Casos de teste

5 Concretização em TestNG

Foram seleccionados 4 casos de sucesso e 4 casos de insucesso da bateria de testes da classe Order (Secção 1).

```java
package supereats.tests;

import org.testng.annotations.*;
import static org.testng.Assert.*;

import supereats.core.*;

public class OrderTest {

  private static final double EPS = 1e-9;

  private Supplier s1;      // fornecedor de referência
  private Supplier s2;      // fornecedor distinto
  private Client   client;

  // Itens do fornecedor s1 (weight em gramas, price inteiro)
  private Item i1;          // preço 10
  private Item i2;          // preço  5
  private Item i50;         // preço 50 (para casos de totalQty)

  // Item de fornecedor distinto (s2)
  private Item iOther;

  @BeforeMethod
  public void setUp() {
    s1     = new Supplier("Loja A");
    s2     = new Supplier("Loja B");
    client = newClient();

    i1     = new Item(100,  10, s1, "Item 1");
    i2     = new Item(200,   5, s1, "Item 2");
    i50    = new Item(150,  50, s1, "Item 50");
    iOther = new Item(100,  10, s2, "Item de outro fornecedor");
  }

  // Helper: assume-se a existência de um construtor de Client.
  private Client newClient() {
```

```
38      return new Client(/* ... dados válidos ... */);
39    }
40
41    // ------------------------------------------------------------------------
42    // Casos de SUCESSO
43    // ------------------------------------------------------------------------
44
45    // TC1: construção válida - todos os invariantes satisfeitos.
46    @Test
47    public void tc1_validConstruction() {
48      Order o = new Order(client, "Rua A", 5000);
49
50      assertEquals(o.totalQuantity(), 0);
51      assertEquals(o.cost(), 0.0, EPS);
52      assertTrue(o.itens().isEmpty());
53    }
54
55    // TC9: addItem numa encomenda vazia com quantidade válida.
56    @Test
57    public void TC9AddItemEmptyOrderValidQuantity() {
58      Order o = new Order(client, "Rua A", 5000);
59
60      o.addItem(i1, 1);
61      assertTrue(o.contains(i1));
62      assertEquals(o.quantity(i1), 1);
63      assertEquals(o.cost(), 10.0, EPS);
64    }
65
66    // TC13: fronteira totalQty <= 20 + floor(cost/50) (On).
67    // Usa-se preco unitario 1 para isolar a fronteira de totalQty sem
68    // violar o limite de custo (I3). Com cost = 24 * 1 = 24, o limite e'
69    // 20 + floor(24/50) = 20 + 0 = 20... pelo que e' preciso elevar o
70    // custo o suficiente para que o limite atinja 24:
71    //   limite = 24  <=>  floor(cost/50) = 4  <=>  cost em [200, 250).
72    // Escolhe-se cost = 240 com itens de preco 10 (24 unidades * 10 = 240),
73    // mantendo cost <= 1000.
74    @Test
75    public void TC13TotalQuantityAtDynamicLimit() {
76      Order o = new Order(client, "Rua A", 5000);
77
78      // 3 itens de preço 10, quantidades 10 + 10 + 4 = 24 unidades
79      Item a = new Item(150, 10, s1, "A");
80      Item b = new Item(150, 10, s1, "B");
81      Item c = new Item(150, 10, s1, "C");
82
83      o.addItem(a, 10);
84      o.addItem(b, 10);
85      o.addItem(c,  4);
86
87      assertEquals(o.totalQuantity(), 24); // = 20 + floor (240/50) = 24
88      assertEquals(o.cost(), 240.0, EPS);
89    }
90
91    // TC19: adicionar item do MESMO fornecedor é aceite.
92    @Test
93    public void TC19SameSupplierIsAccepted() {
94      Order o = new Order(client, "Rua A", 5000);
95
96      o.addItem(i1, 2);
97      o.addItem(i2, 3); // i2 tambem do fornecedor s1
98
99      assertTrue(o.contains(i1));
100     assertTrue(o.contains(i2));
```

```
101       assertEquals(o.totalQuantity(), 5);
102       assertEquals(o.cost(), 2 * 10 + 3 * 5, EPS);
103     }
104
105     // ------------------------------------------------------------------------
106     // Casos de INSUCESSO
107     // ------------------------------------------------------------------------
108
109     // TC2: fronteira addrLength >= 1 (Off) - morada vazia.
110     @Test(expectedExceptions = InvalidOperationException.class)
111     public void TC2EmptyAddressThrows() {
112       new Order(client, "", 5000);
113     }
114
115     // TC6: cliente null.
116     @Test(expectedExceptions = InvalidOperationException.class)
117     public void TC6NullClientThrows() {
118       new Order(null, "Rua A", 5000);
119     }
120
121     // TC10: fronteira quantity <= 10 (Off) - quantidade 11.
122     @Test
123     public void TC10QuantityAboveMaxThrowsAndKeepsState() {
124       Order o = new Order(client, "Rua A", 5000);
125
126       try {
127         o.addItem(i1, 11);
128         fail("Esperava InvalidOperationException");
129       } catch (InvalidOperationException e) {
130         assertFalse(o.contains(i1)); // estado inalterado
131         assertEquals(o.totalQuantity(), 0);
132       }
133     }
134
135     // TC20: adicionar item de fornecedor DISTINTO.
136     @Test(expectedExceptions = InvalidOperationException.class)
137     public void TC20DifferentSupplierThrows() {
138       Order o = new Order(client, "Rua A", 5000);
139
140       o.addItem(i1,     1);    // fornecedor s1
141       o.addItem(iOther, 1);    // fornecedor s2 -> excepção
142     }
143 }
```

Listing 1: Esqueleto da classe de testes TestNG para Order.